

操作系统第六次作业

姓名：陈天乐

班级：无25

学号：

Q1

- 使用 $4\text{KB}=4096\text{B}=2^{12}\text{B}$ 时
 - $20000/4096 = 4\ldots\ldots3616$ ，因此虚拟页号为4，页内偏移为3616
 - $32768/4096 = 8\ldots\ldots0$ ，因此虚拟页号为8，页内偏移为0
 - $60000/4096 = 14\ldots\ldots2656$ ，因此虚拟页号为14，页内偏移为2656
- 使用 $8\text{KB}=8192\text{B}=2^{13}\text{B}$ 时
 - $20000/8192 = 2\ldots\ldots3616$ ，因此虚拟页号为2，页内偏移为3616
 - $32768/8192 = 4\ldots\ldots0$ ，因此虚拟页号为4，页内偏移为0
 - $60000/8192 = 7\ldots\ldots2656$ ，因此虚拟页号为7，页内偏移为2656

Q2

页大小为 $1\text{KB}=1024\text{B}$ 。

逻辑页号	物理页框号
0	4
1	7
2	1
3	2
4	3
5	0

- 逻辑地址5499， $5499/1024 = 5\ldots\ldots379$ ，逻辑页号5对应物理页框号为0，那么物理地址为 $0 \times 1024 + 379 = 379$ 。
- 逻辑地址3746， $3746/1024 = 3\ldots\ldots674$ ，逻辑页号3对应物理页框号为2，那么物理地址为 $2 \times 1024 + 674 = 2722$ 。

Q3

某进程页面访问序列为：2, 3, 2, 1, 5, 2, 4, 5, 3, 2, 5, 2。系统分配给该进程的页框数为3。

- OPT算法

- OPT算法是**选择以后永不使用或者在最长时间内不再被访问的页面进行置换**，访页情况如下：

访问序列	2	3	2	1	5	2	4	5	3	2	5	2
页框	2	2, 3	2, 3	2, 3, 1	2, 3, 5	2, 3, 5	4, 3, 5	4, 3, 5	4, 3, 5	2, 3, 5	2, 3, 5	2, 3, 5
命中情况	M	M	H	M	M	H	M	H	H	M	H	H

缺页率: $\alpha = \frac{6}{12} = 50\%$

- FIFO算法

- FIFO算法是**优先替换最早进入内存的页面**，访页情况如下：

访问序列	2	3	2	1	5	2	4	5	3	2	5	2
页框	2	2, 3	2, 3	2, 3, 1	5, 3, 1	5, 2, 1	5, 2, 4	5, 2, 4	3, 2, 4	3, 2, 4	3, 5, 4	3, 5, 2
命中情况	M	M	H	M	M	M	M	H	M	H	M	M

缺页率: $\alpha = \frac{9}{12} = 75\%$

- CLOCK算法

- CLOCK算法是**选择在最近一段时间内不用的页面予以替换**，访页情况如下：

访问序列	2	3	2	1	5	2	4	5	3	2	5	2
页框	2 ¹	2 ¹ , 3 ¹	2 ¹ , 3 ¹	2 ¹ , 3 ¹ , 1 ¹	3 ⁰ , 1 ⁰ , 5 ¹	1 ⁰ , 5 ¹ , 2 ¹	5 ¹ , 2 ¹ , 4 ¹	5 ¹ , 2 ¹ , 4 ¹	2 ⁰ , 4 ⁰ , 3 ¹	2 ¹ , 4 ⁰ , 3 ¹	2 ¹ , 5 ¹ , 3 ¹	2 ¹ , 5 ¹ , 3 ¹
命中情况	M	M	H	M	M	M	M	H	M	H	M	H

缺页率: $\alpha = \frac{8}{12} = 66.7\%$

- LRU算法
 - LRU算法是**选择在最近一段时间内最久不用的页面予以替换**，访页情况如下：

访问序列	2	3	2	1	5	2	4	5	3	2	5	2
页框	2	2, 3	2, 3	2, 3, 1	2, 5, 1	2, 5, 1	2, 5, 4	2, 5, 4	3, 5, 4	3, 5, 2	3, 5, 2	3, 5, 2
命中情况	M	M	H	M	M	H	M	H	M	M	H	H

缺页率: $\alpha = \frac{7}{12} = 58.3\%$

Q4

页面大小为2KB，程序执行时系统只分配了1个页框。

- 程序A

```
int a[256][256];
int i, j;
for(i = 0; i < 256; i++)
    for(j = 0; j < 256; j++)
        a[i][j] = 0;
```

二维数组a[256][256]每行所占字节数为：256*4=1024字节，每一页是2KB=2048个字节，因此每个页面可以容纳2行数组，数组a占128个页面。因为程序A其循环是对数组逐行扫描，每个页面会缺失1次，故缺失128次。

- 程序B

```
int a[256][256];
int i, j;
for(j = 0; j < 256; j++)
    for(i = 0; i < 256; i++)
        a[i][j] = 0;
```

二维数组a[256][256]每行所占字节数为：256*4=1024字节，每一页是2KB=2048个字节，因此每个页面可以容纳2行数组，数组a占128个页面。因为程序B其循环是对数组逐列扫描，每一列都会导致缺失1次，故缺失256*128=32768次。

Q5

```
int x[N];
int step = M;    //M is some predefined constant
for(int i = 0; i < N; i += step)    x[i] = x[i] + 1;
```

a)

如果该程序运行在页面大小4KB=4096B，且有64个TLB表项。初始时TLB表中还无缓存页，因此欲使得循环的每次执行都会导致TLB缺失，每次访问递增的内存地址必须不小于 4096，故 $step = M = 4096/4 = 1024$ ， $N > 1024$ 即可。

b)

若循环重复多次，TLB 会逐步缓存曾访问过的页（如 0,64,128,...）。那么当64个TLB表项均被缓存后，循环重复时将不再发生缺失，因此，除了 $M = 1024$ 之外，还需使得数组的大小大于4KB*64=256KB，因此 $N \geq 256KB/4B = 65536$ 。